

BÜYÜK DİL MODELLERİ İLE METİNLERDE DUYGU ANALİZİ

Furkan AVCIOĞLU

Yüksek Lisans Öğrencisi, Bursa Uludağ Üniversitesi, Mühendislik Fakültesi, Elektrik-Elektronik Mühendisliği, Bursa, Türkiye.
ORCID: 0009-0002-1976-7028

Prof. Dr. Ersen YILMAZ

Prof. Dr., Bursa Uludağ Üniversitesi, Mühendislik Fakültesi, Elektrik-Elektronik Mühendisliği, Bursa, Türkiye.
ORCID: 0000-0002-6620-655X

ÖZET

Günümüzde teknolojinin gelişmesi ile beraber internet ortamındaki veri ve enformasyon miktarı artış göstermiştir. Büyük veri çaığında insanoğlu verilerden anlamlı bilgi çıkarma amacıyla önemli çalışmalar yapmıştır. Bu çalışma alanlarından birisi de duygu analizidir. Duygu analizinde makine öğrenmesi, derin öğrenme tabanlı algoritmalarla çalışılmaktadır. Duygu analizi finans, savunma sanayii, e-ticaret, lojistik ve sağlık gibi çeşitli sektörlerde kullanılmaktadır. Bu çalışmada winkower/turkish-sentiment-analysis-dataset veriseti kullanılmıştır. Bu veri seti 441 bin eğitim ve 49 bin test verisi olmak üzere pozitif, negatif ve nötr etiketlerine sahip X platformu, vikipedi, mağaza, film, otel yorumlarının birleştirilmesi ile oluşturulmuştur. Bu çalışmada geleneksel makine öğrenmesi modelleri yerine transformatör tabanlı algoritmalar kullanılmıştır. Transformatör tabanlı büyük dil modellerinin seçilmesinin sebebi; geleneksel algoritmaların yaptığı özellik çıkarımı işlemini daha az işlem yükü ile yapmasıdır. Bununla birlikte transformatör tabanlı algoritmalar cümleler arasındaki anlamsal bağlamı da yakalayabilmektedir. Geleneksel makine öğrenmesindeki uzun süren manuel özellik işleme süreçlerinin yerine; transformatör tabanlı modellerde veri alt-sözcük tokenizasyon yöntemi ile bölünerek, kelimeler arasındaki anlamsal bağ öklit uzayında geometrik olarak bulunup, gömme işleminden sonra tokenlere ayrılmış cümle yapılarının aynı uzunluğa getirilmesi için doldurma işlemi yapıp sınıflandırma işlemi tamamlanmaktadır. Bu çalışmada iki BERT tabanlı algoritma ince-ayar ile eğitilerek kullanılmıştır. Artiwise ModernBERT for Turkish algoritmasının ince-ayarlı versiyonunda %96.15 doğruluk, %94.15 makro kesinlik, %92.91 makro duyarlılık, %93.51 makro F1-skor ve %96.12 ağırlıklı F1-skor elde edilmiştir. dbmdz Turkish BERT model algoritmasının ince-ayarlı versiyonunda ise %96.88 doğruluk, %95.25 makro kesinlik, %93.81 makro duyarlılık, %94.50 makro F1-skor elde edilmiştir. Benzetim çalışması her iki algoritmanın da veri seti üzerinde başarılı sonuçlar ürettiğini ve benzer problemlerin çözümde tercih edilebileceklerini göstermiştir.

Anahtar Kelimeler: Duygu analizi, BERT, Büyük Dil Modelleri

SENTIMENT ANALYSIS IN TEXTS WITH LARGE LANGUAGE MODELS

ABSTRACT

With the advancement of technology today, the amount of data and information on the internet has increased. In the era of big data, humanity has made significant efforts to extract meaningful information from data. One of these areas of study is sentiment analysis. Sentiment analysis utilizes machine learning and deep learning-based algorithms. Sentiment analysis is used in various sectors such as finance, defense industry, e-commerce, logistics, and healthcare. The winkower/turkish-sentiment-analysis-dataset was used in this study. This dataset was created by combining X platform, Wikipedia, store, movie, and hotel reviews labeled as positive, negative, and neutral, consisting of 441,000 training and 49,000 test data points. Transformer-based algorithms were used in this study instead of traditional machine learning models. The reason for choosing transformer-based large language models is that they perform feature extraction with less computational load than traditional algorithms.

Furthermore, transformer-based algorithms can also capture the semantic context between sentences. Instead of the lengthy manual feature processing involved in traditional machine learning, transformer-based models divide the data using a subword tokenization method. The semantic relationship between words is then found geometrically in Euclidean space. After embedding, padding is performed to bring the tokenized sentence structures to the same length, and the classification process is completed. In this study, two BERT-based algorithms were trained with fine-tuning and used. The fine-tuned version of the Artiwise ModernBERT for Turkish algorithm achieved 96.15% accuracy, 94.15% macro precision, 92.91% macro recall, 93.51% macro F1-score, and 96.12% weighted F1-score. The fine-tuned version of the dbmdz Turkish BERT model algorithm achieved 96.88% accuracy, 95.25% macro precision, 93.81% macro recall, and 94.50% macro F1-score. The simulation study showed that both algorithms produced successful results on the dataset and could be preferred for solving similar problems.

Keywords: Sentiment analysis, BERT, Large Language Models

GİRİŞ

Günümüz dünyasında teknolojinin hızla ilerlemesi ile beraber internet ortamındaki veri ve enformasyonun miktarı önemli ölçüde artış göstermiştir(Khan et al., 2014). Büyük ölçüdeki bu veri ve enformasyon artışı araştırmacıları bu konuda bilimsel çalışmalar yapmaya, internette bulunan bu verilerden anlamlı bilgiler çıkarmaya itmiştir(Zhuang et al., 2017). Bu verilerden elde edilen anlamlı bilgiler ticari faaliyet gösteren firmalara ticari kâr sağlamış, bilim insanlarına da literatüre katkı sağlama imkanı vermiştir. Verilerden anlamlı bilgi çıkarma noktasında internet dünyasında çeşitli veri tiplerini gözlemlemekteyiz. Resim, tablo, metin, ses ve sinyal gibi farklı veri tipleri internet ortamında yer almaktadır(Najafabadi et al., 2015).

Sosyal medya kullanımının artması ile birlikte kullanıcılar birbirlerine internet ortamından paylaşımlar gönderip, yorum yapmaktadır(Poecze et al., 2018). Bu yorumların gerçek bir birey tarafından yazılıp yazılmadığı, yorum yazan bireyin o yorumu yazarkenki duygu durumu önem arz etmektedir(Alonso et al., 2021). Bu noktada metinlerin duygu durumunu belirleme işlemine duygu analizi işlemi denilmektedir(Medhat et al., 2014). Duygu analizi, doğal dil işleme alanının en önemli alt alanlarından birisidir. Duygu analizi, bankacılık, sigortacılık, savunma sanayii, lojistik ve sağlık gibi çeşitli sektörlerde ihtiyaç duyulan bir çalışma alanıdır(Chutia & Baruah, 2024).

Duygu analizi esnasında yapılan işlemleri özümsemek için yapay zekanın alt dalı olan makine öğrenmesindeki veri ön işleme, model geliştirme, model metrikleri değerlendirme gibi konularda bilgi sahibi olunması şiddetle önem arz etmektedir(Schröer et al., 2021). Makine öğrenmesi alanında veri tipi metin, ses, görüntü, tablo verisi olabilmektedir. Veri tipinden bağımsız olarak bir veri bilimi proje döngüsünde öncelikle veri temizlenmeli, veri format değişiklikleri yapılmalıdır(Zelaya, 2019). Sonrasında uygun makine öğrenmesi algoritması ile veriseti üzerinde eğitilme gerçekleştirilerek başarımlar sonuçları probleme özgü başarımlar metrikleri ile sunulmalıdır(Rainio et al., 2024). Aşırı öğrenme veya düşük öğrenme problemi yaşanırsa model eğitimi kısmında hiperparametre optimizasyonu yapılarak başarımlar sonuçları tekrar incelenmelidir(Van Der Aalst et al., 2010).

Doğal dil işlemenin önemli bir alt dalı olan duygu analizi alanında araştırmacılar tarafından önemli algoritmalar geliştirilmiştir. Bu algoritmalar gelişim sırasıyla kural bazlı algoritmalar, istatistiksel algoritmalar, frekans bazlı metin vektörizasyon algoritmaları, bağlam bağımsız durgun gömme modelleri, bağlamaştırıcı derin öğrenme tabanlı gömme modelleridir(Camacho-Collados & Taher Pilehvar, 2020). Bu algoritmalarından en başarılı olanı anlam ve bağlamı yakalayan transformatör tabanlı algoritmalar(Tahmid et al., 2020). Kural tabanlı algoritmalar veriler ve öğrenme sürecinde uzun kural yapılarını ve ön işleme süreçlerini gerektirmektedir(JeeHee et al., 2019). İstatistiksel algoritmalar ise uzun kural yapısı gerekmesek bile kelimeler arasındaki bağlam yakalanamamakta ve uzun manuel ön işleme süreçleri araştırmacıları uğraştırmaktadır. Bağlam bağımsız durgun gömme modellerinde de anlam yakalanabilmekte fakat bağlam yakalama işleminde problemler yaşanmaktadır(Wahyu Trisna et al., 2024).

Bu çalışmada bağlamaştırıcı derin öğrenme modellerinden olan transformatör tabanlı modeller kullanılmıştır. Transformatör tabanlı dil modellerinin seçilmesindeki en önemli sebep, semantik anlamı metinden öz-dikkat mekanizması sayesinde yakalamasıdır(Malik et al., 2023).

Bu çalışmada BERT tabanlı transformatör temelli dil modelleri ile çalışılmış ve winkower/turkish-sentiment-analysis-dataset veriseti kullanılmıştır. Yapılan deneyler sonucunda kullanılan dil modelleri ile başarılı sonuçlar elde edilmiştir.

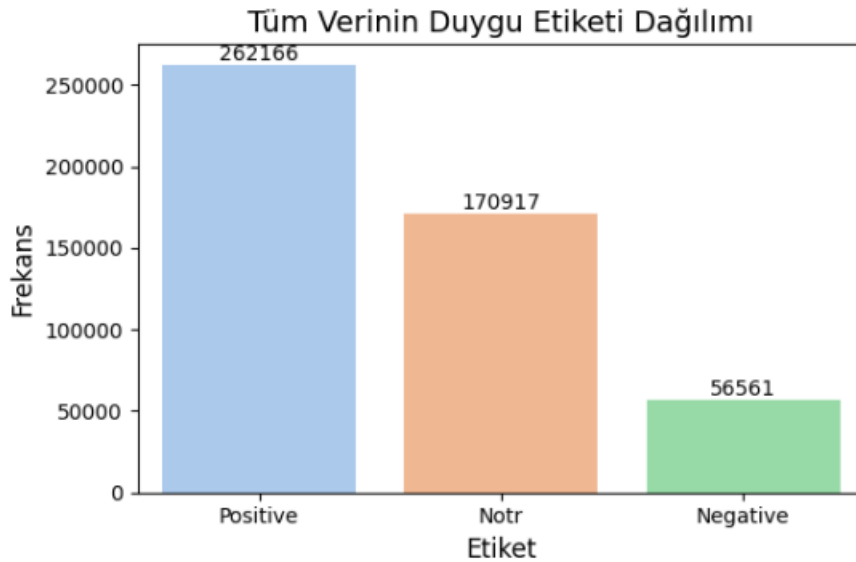
MATERYAL VE YÖNTEM

Transformatör Tabanlı Dil Modelleri ile Duygu Analizi

Bu çalışmanın amacı transformatör tabanlı BERT dil modelleri ile metinlerde duygu analizi gerçekleştirmektir. Büyük veri setlerinde uzun ön işleme süreçlerine takılmadan, metinlerdeki anlam ve bağlam bilgisini kaliteli bir şekilde yakalayarak, genelleştirilebilir, öznetelik çıkarımını otomatik olarak yapabilen derin öğrenme tabanlı transformatör modelleri geleneksel modeller yerine tercih edilmektedir(Maslennikova, 2019).

Veri Seti

Bu çalışmada winkower/turkish-sentiment-analysis-dataset veriseti kullanılmıştır. Bu veri seti 441 bin eğitim ve 49 bin test verisi olmak üzere pozitif, negatif ve nötr etiketlerine sahip X platformu, vikipedi, mağaza, film ve otel yorumlarının birleştirilmesi ile oluşturulmuştur. Farklı sektörlerden verilerin toplanması modelin genelleştirilebilir olmasını sağlamakta ve farklı problem uzaylarında başarı göstermesinde önemli katkılar sağlamaktadır (Barbiero et al., 2020). Şekil 1’de verinin etiket dağılımı gösterilmiştir.



Şekil 1. Veri Etiket Dağılımı

Önerilen Çözüm – Uygulama Adımları

Bu çalışma Kaggle ortamında geliştirilmiştir. Bu çalışmada iki farklı BERT tabanlı transformatör model kullanılmıştır. Kaggle ortamında ilk adım olarak makine öğrenmesi, derin öğrenme ve doğal dil işleme kütüphaneleri yüklenmiştir (Şekil 2).

```
BEGIN
  INSTALL packages quietly (without showing verbose output):
    - datasets: Hugging Face datasets library
    - transformers[torch]: Transformers library with PyTorch dependencies
    - evaluate: Hugging Face evaluation library
    - huggingface-hub: Hugging Face Hub client library
    - scikit-learn: Machine learning library
    - matplotlib: Plotting and visualization library

  NOTE: The -q flag suppresses installation output and progress bars
  NOTE: [torch] specifies the PyTorch backend for transformers
END
```

Şekil 2. Kütüphane Yükleme

Daha sonraki adım olarak Hugging Face arayüzü ile Kaggle entegrasyonu sağlanmıştır. Şekil 3’de bu işlemin sözde kodları gösterilmektedir.

```
BEGIN HuggingFace_Login_Procedure

  // Step 1: Define authentication token
  SET hf_token = "your_huggingface_token_here"

  // Step 2: Authenticate with Hugging Face Hub
  CALL huggingface_hub.login(token = hf_token)

  // Step 3: Verification (implied)
  IF authentication_successful THEN
    PRINT "Successfully logged in to Hugging Face Hub"
    SET authenticated = TRUE
  ELSE
    PRINT "Authentication failed"
    SET authenticated = FALSE
  END IF

END HuggingFace_Login_Procedure
```

Şekil 3. Hugging Face-Kaggle Entegrasyonu

Hugging Face-Kaggle entegrasyonundan sonra çalışmada ihtiyaç duyulan kütüphaneler projeye eklenmiştir. Bu kütüphaneler veri ön işleme, derin öğrenme modelleri, model başarı metrikleri ve veri görselleştirme işlemleri için gereklidir(Nguyen et al., 2019). Şekil 4’de çalışmada kullanılan kütüphanelerin içe aktarımı gösterilmiştir.

```
BEGIN

IMPORT load_dataset FROM datasets
IMPORT AutoTokenizer FROM transformers
IMPORT AutoModelForSequenceClassification FROM transformers
IMPORT TrainingArguments FROM transformers
IMPORT Trainer FROM transformers
IMPORT DataCollatorWithPadding FROM transformers
IMPORT evaluate
IMPORT numpy AS np
IMPORT torch
IMPORT confusion_matrix FROM sklearn.metrics
IMPORT classification_report FROM sklearn.metrics
IMPORT pyplot AS plt FROM matplotlib
IMPORT sns FROM seaborn

END
```

Şekil 4. Kütüphane İçe Aktarma

Bu çalışmada daha dinamik konfigürasyon işlemi için CONFIG adında bir sözlük yapısı Python ortamında oluşturulmuştur. Tablo 1’de konfigürasyon parametrelerinin açıklaması yer almaktadır. Konfigürasyon işleminden sonra her çalıştırmada aynı çıktının alınması için sabit bir seed parametresi uygulanmıştır. Şekil 5 ve Şekil 6’da konfigürasyon parametreleri ve sabit kod çıktısı alma işlemi görülmektedir(Dutta et al., 2022).

Tablo 1. Konfigürasyon Parametreleri

Konfigürasyon Parametre İsimleri	BERTurk	ModernBERT Turkish	Konfigürasyon Parametre Açıklamaları
model_name	dbmdz/bert-base-turkish-cased	artiwise-ai/modernbert-base-tr-uncased	Kullanılacak modelin adı
dataset_name	winvoker/turkish-sentiment-analysis-dataset	winvoker/turkish-sentiment-analysis-dataset	Kullanılacak veri setinin adı
output_dir	turkish-sentiment3	turkish-sentiment4	Eğitilen modelin kaydedileceği dizin
hub_model_id	dexter231/turkish-sentiment3	dexter231/turkish-sentiment4	Modelin Hugging Face ortamında oluşturulacak ismi
learning_rate	3e-5	5e-5	Model ağırlıklarının öğrenme hızı
per_device_train_batch_size	64	16	Eğitim işleminde her cihazda kullanılacak örnek miktarı
per_device_eval_batch_size	128	16	Validasyon işleminde her cihazda kullanılacak örnek miktarı

num_train_epochs	4	-	Modelin tüm eğitim verisi üzerinde tekrarlanma sayısı
max_steps	1600	2000	Maksimum eğitim adım sayısı
max_seq_lenght	128	128	Metinlerin maksimum token uzunluğu
seed	42	42	Rassallık parametresi
warmup_steps	400	400	Öğrenme oranının arttığı baş değer
optimizer	adamw_torch	adamw_torch	Optimizasyon algoritması
lr_scheduler_type	cosine	linear	Öğrenme oranını gerektiği zaman düşüren düzenleyici
push_to_hub	True	True	Modelin eğitim sonunda kayıt edileceğini düzenleyen parametre
adam_beta1	0.9	0.9	Optimizasyon algoritmasının birinci moment tahmini için beta değeri
adam_beta2	0.999	0.999	Optimizasyon algoritmasının ikinci moment tahmini için beta değeri
adam_epsilon	1e-08	1e-08	Optimizasyon algoritmasının kararlılık için kullandığı değer

```
ALGORITHM: Model Training Configuration Setup

BEGIN
  // Define configuration parameters
  CONFIG = {
    model_name: "artiwise-ai/modernbert-base-tr-uncased"
    dataset_name: "winvoker/turkish-sentiment-analysis-dataset"
    output_dir: "turkish-sentiment4"
    hub_model_id: "dexter231/turkish-sentiment4"
    learning_rate: 0.00005
    per_device_train_batch_size: 16
    per_device_eval_batch_size: 16
    max_steps: 2000
    max_seq_length: 128
    seed: 42
    warmup_steps: 400
    optimizer: "adamw_torch"
    lr_scheduler_type: "linear"
    push_to_hub: TRUE
    adam_beta1: 0.9
    adam_beta2: 0.999
    adam_epsilon: 0.00000001
  }

  // Set random seed for reproducibility
  SET torch.random_seed TO CONFIG["seed"]
  SET numpy.random_seed TO CONFIG["seed"]

END ALGORITHM
```

Şekil 5. Konfigürasyon Parametreleri-ModernBERT Turkish

```
BEGIN Configuration Setup

  // Define configuration parameters
  SET CONFIG = {
    model_name: "dbmdz/bert-base-turkish-cased",
    dataset_name: "winvoker/turkish-sentiment-analysis-dataset",
    output_dir: "turkish-sentiment3",
    hub_model_id: "dexter231/turkish-sentiment3",
    learning_rate: 0.00003,
    per_device_train_batch_size: 64,
    per_device_eval_batch_size: 128,
    num_train_epochs: 4,
    max_steps: 1600,
    max_seq_length: 128,
    seed: 42,
    warmup_steps: 400,
    optimizer: "adamw_torch",
    lr_scheduler_type: "cosine",
    push_to_hub: True,
    adam_beta1: 0.9,
    adam_beta2: 0.999,
    adam_epsilon: 0.00000001
  }

  // Set random seeds for reproducibility
  SET torch.random_seed TO CONFIG["seed"]
  SET numpy.random_seed TO CONFIG["seed"]

END Configuration Setup
```

Şekil 6. Konfigürasyon Parametreleri- BERTurk

Bu işlemlerden sonra `load_dataset` fonksiyonu ile veriseti içe dahil edilmektedir. Verinin label kısmında yer alan Positive, Nötr ve Negative metin kolonları derin öğrenme modelinde işlenebilmesi için numerik hale getirilmektedir (Yu et al., 2022). Veriyi aktarılması ve kategorik veriyi numerik veriye çevirme işlemi Şekil 7’de gösterilmektedir.

```
BEGIN Load and Prepare Dataset

// Step 1: Load dataset
PRINT "📄 Loading dataset..."
dataset = LOAD_DATASET(CONFIG["dataset_name"])

// Step 2: Define label mapping
label_mapping = {
    "Positive": 0,
    "Notr": 1,
    "Negative": 2
}

// Step 3: Apply label mapping transformation
FOR EACH example IN dataset DO
    example["label"] = label_mapping[example["label"]]
END FOR

// Dataset is now prepared with numeric labels

END Load and Prepare Dataset
```

Şekil 7. Veri Yükleme ve Kategorik Veriyi Numerik Veriye Çevirme

Aktarılan verisetinde eğitilecek veri tipi metindir. Metin verisi ham hali eğitilememektedir. Bu yüzden metne alt-kelime tokenizasyon işlemi uygulanmaktadır. Alt-kelime tokenizasyon işlemindeki amaç metin tipindeki veriyi sayısal olarak temsil edilebilir hale getirmektir (Nayak et al., 2020). Şekil 8’de çalışmadaki tokenizasyon işlemi görülmektedir.

```
BEGIN

// 2. Load tokenizer
PRINT "📄 Loading tokenizer..."
tokenizer = LOAD_TOKENIZER_FROM_PRETRAINED(CONFIG["model_name"])

// 3. Tokenize data function
FUNCTION tokenize_function(examples)
    RETURN tokenizer(
        text: examples["text"],
        truncation: TRUE,
        max_length: CONFIG["max_seq_length"],
        padding: FALSE
    )
END FUNCTION

END
```

Şekil 8. Tokenizasyon İşlemi

Veriye alt-kelime tokenizasyon işlemi uyguladıktan sonra metne dinamik padding işlemi yapılmaktadır(Zhang et al., 2021). Bu işlem Şekil 9’da görüldüğü biçimde gerçekleştirilmiştir.

```
BEGIN
  // Initialize data collator with padding functionality
  FUNCTION create_data_collator(tokenizer):
    SET data_collator = DataCollatorWithPadding(
      tokenizer: tokenizer,
      padding: True,
      max_length: Optional[int] // Could be specified
    )
    RETURN data_collator

  // Usage
  data_collator = create_data_collator(tokenizer)
END
```

Şekil 9. Dinamik Padding İşlemi

Duygu analizi işleminde temel amaç pozitif, negatif, nötr olan veriyi kaliteli bir şekilde sınıflandırmaktır. Veri ön işleme işlemlerinden sonra metin sınıflandırma konfigürasyonu yapılmaktadır (Şekil 10).

```
BEGIN
  PRINT "🚀 Loading model..."

  SET model = AutoModelForSequenceClassification.from_pretrained(
    PARAMETERS:
      model_name = CONFIG["model_name"],
      num_labels = 3,
      id2label = {
        0: "Positive",
        1: "Neutral",
        2: "Negative"
      },
      label2id = {
        "Positive": 0,
        "Neutral": 1,
        "Negative": 2
      }
  )
END
```

Şekil 10. Metin Sınıflandırma Konfigürasyonu

Model başarımının değerlendirilmesinde doğruluk, kesinlik, duyarlılık ve F1-skor metrikleri kullanılmıştır (Şekil 11). Dengeli olmayan verilerde doğrulukla beraber kesinlik ve duyarlılık metrikleri modeli bilimsel metodolojiye göre daha uygun değerlendirmeyi sağlamaktadır(Padurariu & Breaban, 2019).

```
BEGIN FUNCTION compute_metrics(eval_pred)
    // Unpack predictions and labels from eval_pred tuple
    SET predictions = eval_pred[0]
    SET labels = eval_pred[1]

    // Convert prediction probabilities to class labels by taking argmax
    SET predictions = np.argmax(predictions, axis=1)

    // Initialize empty dictionary to store results
    SET results = {}

    // Calculate Accuracy
    SET accuracy_results = accuracy_metric.compute(predictions=predictions, references=labels)
    SET results["accuracy"] = accuracy_results["accuracy"]

    // Calculate F1 Macro Score
    SET f1_macro = f1_metric.compute(predictions=predictions, references=labels, average="macro")
    SET results["f1_macro"] = f1_macro["f1"]

    // Calculate F1 Weighted Score
    SET f1_weighted = f1_metric.compute(predictions=predictions, references=labels, average="weighted")
    SET results["f1_weighted"] = f1_weighted["f1"]

    // Calculate Precision Macro Score
    SET precision_metric = evaluate.load("precision")
    SET precision_macro = precision_metric.compute(predictions=predictions, references=labels, average="macro")
    SET results["precision_macro"] = precision_macro["precision"]

    // Calculate Recall Macro Score
    SET recall_metric = evaluate.load("recall")
    SET recall_macro = recall_metric.compute(predictions=predictions, references=labels, average="macro")
    SET results["recall_macro"] = recall_macro["recall"]

    // Return the compiled results dictionary
    RETURN results
END FUNCTION
```

Şekil 11. Model Başarı Metrikleri

Hiperparametre optimizasyonu model eğitimindeki en önemli aşamalarındandır. Hiperparametre optimizasyonu ile model daha hızlı, daha tutarlı hale gelmektedir(A İlemobayo et al., 2024). Şekil 12’de hiperparametre ayarları ve Tablo 1’de ise hiperparametrelerin detayları gösterilmektedir.

```
BEGIN
    SET training_args = NEW TrainingArguments WITH PARAMETERS:
        output_dir = CONFIG["output_dir"]
        learning_rate = CONFIG["learning_rate"]
        per_device_train_batch_size = CONFIG["per_device_train_batch_size"]
        per_device_eval_batch_size = CONFIG["per_device_eval_batch_size"]
        max_steps = CONFIG["max_steps"]
        warmup_steps = CONFIG["warmup_steps"]
        eval_strategy = "steps"
        eval_steps = 200
        save_strategy = "steps"
        save_steps = 200
        load_best_model_at_end = True
        metric_for_best_model = "f1"
        logging_dir = "./logs"
        logging_steps = 100
        report_to = "none"
        push_to_hub = CONFIG["push_to_hub"]
        hub_model_id = CONFIG["hub_model_id"]
        hub_strategy = "end"
        seed = CONFIG["seed"]
        greater_is_better = True
        optim = CONFIG["optimizer"]
        lr_scheduler_type = CONFIG["lr_scheduler_type"]
        # AdamW optimizer specific parameters
        adam_beta1 = CONFIG["adam_beta1"]
        adam_beta2 = CONFIG["adam_beta2"]
        adam_epsilon = CONFIG["adam_epsilon"]
END
```

Şekil 12. Hiperparametre Ayarları

Şekil 13’de model eğitim işleminin uygulamalı hali gösterilmektedir. Şekil 14’de BERTurk eğitim süreci, Şekil 15’de ise ModernBERT Turkish eğitim süreci gösterilmektedir.

```
# 8. Create Trainer
BEGIN
  INITIALIZE trainer WITH:
    model = model
    training_args = training_args
    train_dataset = tokenized_dataset["train"]
    eval_dataset = tokenized_dataset["test"]
    data_collator = data_collator
    tokenizer = tokenizer
    compute_metrics = compute_metrics
  END INITIALIZE
END

# 9. Train
BEGIN
  PRINT "🔥 Starting training..."
  PRINT "Model: {CONFIG['model_name']}"
  PRINT "Training steps: {CONFIG['max_steps']}"
  PRINT "Batch size: {CONFIG['per_device_train_batch_size']}"
  PRINT "Learning rate: {CONFIG['learning_rate']}"
  PRINT "Warmup steps: {CONFIG['warmup_steps']}"
  PRINT "Optimizer: {CONFIG['optimizer']}"
  PRINT "Scheduler: {CONFIG['lr_scheduler_type']}"

  CALL trainer.train()
END
```

Şekil 13. Model Eğitim İşlemi

🔥 Starting training...

Model: dbmdz/bert-base-turkish-cased
Training steps: 1600
Batch size: 64
Learning rate: 3e-05
Warmup steps: 400
Optimizer: adamw_torch
Scheduler: cosine

[1600/1600 43:22, Epoch 0/1]															
Step	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall	Positive F1	Positive Precision	Positive Recall	Neutral F1	Neutral Precision	Neutral Recall	Negative F1	Negative Precision	Negative Recall
200	0.226200	0.139776	0.948453	0.915220	0.908393	0.922750	0.952493	0.962641	0.942556	0.992893	0.988631	0.997192	0.800273	0.773906	0.828501
400	0.126500	0.112540	0.959032	0.927680	0.936914	0.919456	0.962634	0.954547	0.970859	0.996315	0.996140	0.996490	0.824093	0.860054	0.791018
600	0.121300	0.127044	0.954519	0.911806	0.957275	0.883421	0.959255	0.930726	0.989587	0.996113	0.995269	0.996958	0.780052	0.945830	0.663720
800	0.099500	0.117069	0.960625	0.933940	0.928407	0.939943	0.963562	0.968951	0.958233	0.996749	0.997889	0.995612	0.841508	0.818379	0.865983
1000	0.108000	0.094379	0.966690	0.941322	0.951309	0.932499	0.969725	0.962779	0.976771	0.996412	0.993543	0.999298	0.857829	0.897604	0.821429
1200	0.095600	0.094397	0.965526	0.941769	0.936900	0.946954	0.967941	0.973751	0.962200	0.997605	0.996150	0.999064	0.859760	0.840798	0.879597
1400	0.097100	0.087407	0.968835	0.944856	0.953262	0.937245	0.971317	0.964288	0.978449	0.998099	0.997778	0.998420	0.865152	0.897719	0.834866
1600	0.088700	0.087576	0.968814	0.944972	0.952468	0.938117	0.971276	0.964986	0.977648	0.998099	0.997720	0.998479	0.865541	0.894697	0.838225

Şekil 14. BERTurk Eğitim Süreci

🔥 Starting training...

Model: artiwise-ai/modernbert-base-tr-uncased

Training steps: 2000

Batch size: 16

Learning rate: 5e-05

Warmup steps: 400

Optimizer: adamw_torch

Scheduler: linear

[2000/2000 41:49, Epoch 0/1]

Step	Training Loss	Validation Loss	Accuracy	F1 Macro	F1 Weighted	Precision Macro	Recall Macro
200	0.198700	0.165448	0.939998	0.907417	0.941318	0.894406	0.923572
400	0.197800	0.165294	0.940672	0.909164	0.942736	0.893756	0.930283
600	0.144400	0.157241	0.951966	0.915651	0.950223	0.942479	0.895656
800	0.144600	0.125163	0.956030	0.923896	0.954912	0.940721	0.910132
1000	0.106700	0.138883	0.957439	0.929385	0.957390	0.931274	0.927541
1200	0.133300	0.124375	0.958726	0.927854	0.957603	0.946430	0.912827
1400	0.093500	0.129731	0.959583	0.933627	0.959802	0.930906	0.936467
1600	0.107200	0.119628	0.960380	0.930892	0.959346	0.949651	0.915634
1800	0.123500	0.112109	0.961319	0.933814	0.960702	0.944950	0.924021
2000	0.109700	0.108327	0.961503	0.935105	0.961158	0.941538	0.929139

Şekil 15. ModernBERT Turkish Eğitim Süreci

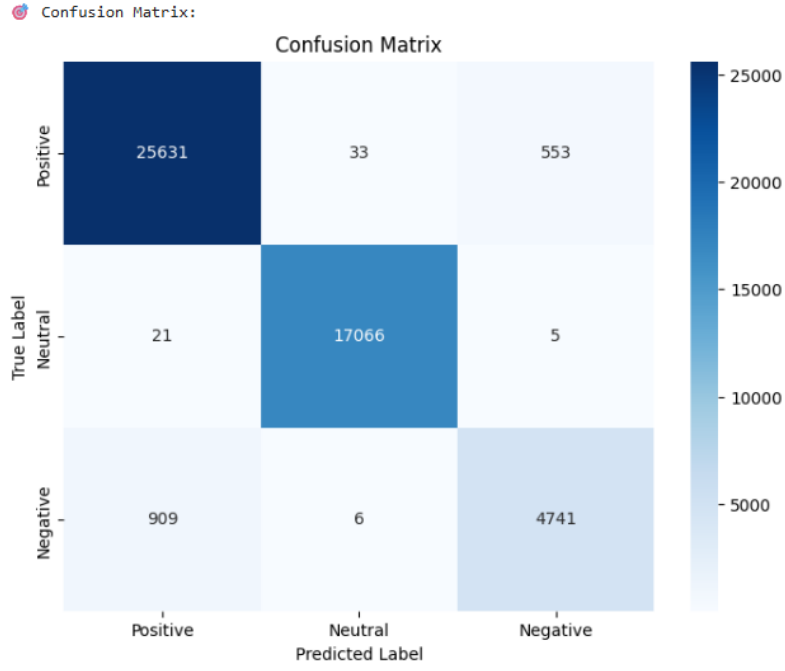
Son olarak eğitilen modeller Hugging Face platformuna kaydedilmektedir (Şekil 16).

```
BEGIN
  IF CONFIG["push_to_hub"] IS TRUE THEN
    PRINT "📦 Pushing model to Hub..."
    CALL trainer.push_to_hub()
    PRINT "✅ Model pushed to: " + CONFIG['hub_model_id']
  END IF
END
```

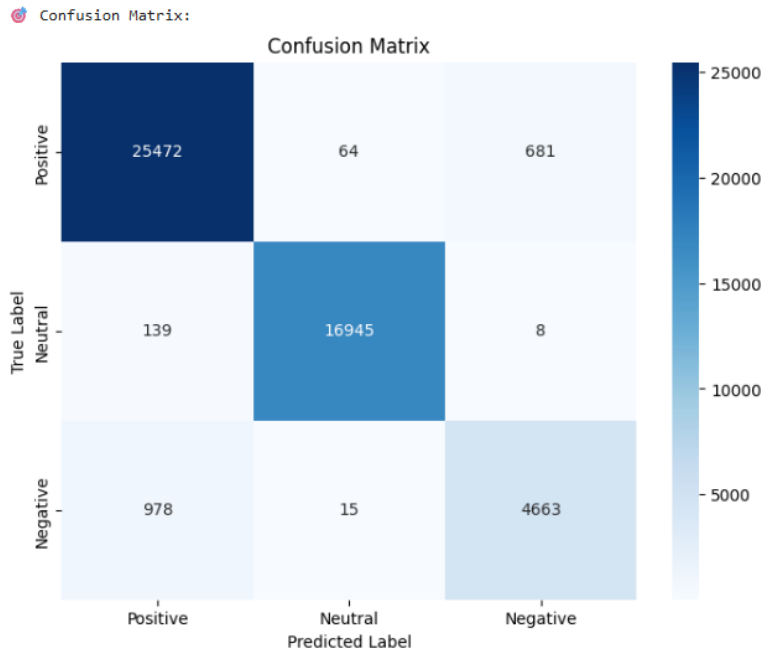
Şekil 16. Model Kaydetme

BULGULAR VE TARTIŞMA

Bu çalışmada BERTurk ve ModernBERT Turkish algoritmaları ince-ayar ile eğitilerek seçilen veriseti üzerinde kullanılmıştır. Algoritmalar için karmaşıklık matrisleri Şekil 17 ve Şekil 18’de sunulmuştur. Her iki modelin başarımları ile aynı veriseti üzerinde gerçekleştirilmiş ve literatürde öne çıkan iki farklı çalışmasının sonuçlarının yer aldığı karşılaştırma tablosu Tablo 2’de sunulmuştur.



Şekil 17. BERTurk Karmaşıklık Matrisi



Şekil 18. ModernBERT Turkish Karmaşıklık Matrisi

Tablo 2. Başarım Karşılaştırması

Algoritma	Birinci Yazar	Doğruluk	Kesinlik	Duyarlılık	F1-Skor
BERTurk	Avcıoğlu, F. (Bu çalışma)	%96.88	%95.25	%93.81	%94.50
ModernBERT Turkish	Avcıoğlu, F. (Bu çalışma)	%96.15	%94.15	%92.91	%93.51
CNN- Transformatör	Görmez, Y.(Görmez et al., 2024)	%94.64	%92.63	%90,43	%91,44
XLM-RoBERTa -XGBoost	Arslan, R. S.(Arslan, 2024)	%93.60	%93.30	%93,40	%93,70

SONUÇ

Bu çalışmada iki BERT tabanlı algoritma ince-ayar ile eğitilerek kullanılmıştır. Artwise ModernBERT for Turkish algoritmasının ince-ayarlı versiyonunda %96.15 doğruluk, %94.15 makro kesinlik, %92.91 makro duyarlılık, %93.51 makro F1-skor ve %96.12 ağırlıklı F1-skor elde edilmiştir. dbmdz Turkish BERT model algoritmasının ince-ayarlı versiyonunda ise %96.88 doğruluk, %95.25 makro kesinlik, %93.81 makro duyarlılık, %94.50 makro F1-skor elde edilmiştir. Benzetim çalışması her iki algoritmanın da veriseti üzerinde başarılı sonuçlar ürettiğini ve benzer problemlerin çözümde tercih edilebileceklerini göstermiştir.

KAYNAKÇA

- A Ilemobayo, J., Durodola, O., Alade, O., J Awotunde, O., T Olanrewaju, A., Falana, O., Ogungbire, A., Osinuga, A., Ogunbiyi, D., Ifeanyi, A., E Odezuligbo, I., & E Edu, O. (2024). Hyperparameter Tuning in Machine Learning: A Comprehensive Review. *Journal of Engineering Research and Reports*, 26(6), 388–395. <https://doi.org/10.9734/jerr/2024/v26i61188>
- Alonso, M. A., Vilares, D., Gómez-Rodríguez, C., Vilares, J., & Lys, G. (2021). *electronics Sentiment Analysis for Fake News Detection*. <https://doi.org/10.3390/electronics>
- Arslan, R. S. (2024). Turkish text sentiment analysis based on transformer models and machine learning. *VI. Uluslararası Türk Dünyası Fen Bilimleri ve Mühendislik Kongresi (TURK-COSE 2024)*, 101-109.
- Barbiero, P., Squillero, G., & Tonda, A. (2020). *Modeling Generalization in Machine Learning: A Methodological and Computational Study*. <http://arxiv.org/abs/2006.15680>
- Camacho-Collados, J., & Taher Pilehvar, M. (2020). *Embeddings in Natural Language Processing*.
- Chutia, T., & Baruah, N. (2024). A review on emotion detection by using deep learning techniques. *Artificial Intelligence Review*, 57(8). <https://doi.org/10.1007/s10462-024-10831-1>
- Dutta, S., Arunachalam, A., & Misailovic, S. (2022). To Seed or Not to Seed? An Empirical Analysis of Usage of Seeds for Testing in Machine Learning Projects. *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 151–161. <https://doi.org/10.1109/ICST53961.2022.00026>
- Görmez, Y., Arslan, H., & Atak, B. (2024). Türkçe Metinlerde Duygu Analizi: Derin Öğrenme Yaklaşımlarının ve Ön İşlem Süreçlerinin Model Performansına Etkisi. *Fırat Üniversitesi Mühendislik Bilimleri Dergisi*, 36(1), 509–520. <https://doi.org/10.35234/fumbd.1429040>
- JeeHee, L., June-Seong, Y., & JeongWook, S. (2019). Development of Automatic-Extraction Model of Poisonous Clauses in International Construction Contracts Using Rule-Based NLP. *Journal of Computing in Civil Engineering*, 33(3), 04019003. [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000807](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000807)
- Khan, N., Yaqoob, I., Hashem, I. A. T., Inayat, Z., Mahmoud Ali, W. K., Alam, M., Shiraz, M., & Gani, A. (2014). Big Data: Survey, Technologies, Opportunities, and Challenges. *The Scientific World Journal*, 2014(1), 712826. <https://doi.org/https://doi.org/10.1155/2014/712826>

- Malik, Muhammad Shahid Iqbal, Nawaz, Aftab, Jamjoom, Mona Mamdouh, & Ignatov, Dmitry I. (2023). Effectiveness of ELMo embeddings, and semantic models in predicting review helpfulness. *Intelligent Data Analysis*, 28(4), 1045–1065. <https://doi.org/10.3233/IDA-230349>
- Maslennikova, E. (2019). *ELMo Word Representations For News Protection*.
- Medhat, W., Hassan, A., & Korashy, H. (2014). Sentiment analysis algorithms and applications: A survey. *Ain Shams Engineering Journal*, 5(4), 1093–1113. <https://doi.org/10.1016/J.ASEJ.2014.04.011>
- Najafabadi, M. M., Villanustre, F., Khoshgoftaar, T. M., Seliya, N., Wald, R., & Muharemagic, E. (2015). Deep learning applications and challenges in big data analytics. *Journal of Big Data*, 2(1). <https://doi.org/10.1186/s40537-014-0007-7>
- Nayak, A., Timmapathini, H. P., Ponnalagu, K., & Venkoparao, V. (2020). Domain adaptation challenges of BERT in tokenization and sub-word representations of Out-of-Vocabulary words. In *Online. Association for Computational Linguistics*. <https://github.com/>
- Nguyen, G., Dlugolinsky, S., Bobák, M., Tran, V., López García, Á., Heredia, I., Malík, P., & Hluchý, L. (2019). Machine Learning and Deep Learning frameworks and libraries for large-scale data mining: a survey. *Artificial Intelligence Review*, 52(1), 77–124. <https://doi.org/10.1007/s10462-018-09679-z>
- Padurariu, C., & Breaban, M. E. (2019). Dealing with Data Imbalance in Text Classification. *Procedia Computer Science*, 159, 736–745. <https://doi.org/10.1016/J.PROCS.2019.09.229>
- Poeche, F., Ebster, C., & Strauss, C. (2018). Social media metrics and sentiment analysis to evaluate the effectiveness of social media posts. *Procedia Computer Science*, 130, 660–666. <https://doi.org/10.1016/J.PROCS.2018.04.117>
- Rainio, O., Teuho, J., & Klén, R. (2024). Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14(1), 6086. <https://doi.org/10.1038/s41598-024-56706-x>
- Schröer, C., Kruse, F., & Gómez, J. M. (2021). A Systematic Literature Review on Applying CRISP-DM Process Model. *Procedia Computer Science*, 181, 526–534. <https://doi.org/10.1016/J.PROCS.2021.01.199>
- Tahmid, M., Laskar, R., Huang, J., & Hoque, E. (2020). *Contextualized Embeddings based Transformer Encoder for Sentence Similarity Modeling in Answer Selection Task*. <https://webscope.sandbox.yahoo.com/>
- Van Der Aalst, W. M. P., Rubin, V., Verbeek, H. M. W., Van Dongen, B. F., Kindler, E., & Günther, C. W. (2010). Process mining: A two-step approach to balance between underfitting and overfitting. *Software and Systems Modeling*, 9(1), 87–111. <https://doi.org/10.1007/s10270-008-0106-z>
- Wahyu Trisna, K., Huang, J., Lei, H., & Muntina Dharma, E. (2024). FROM CONTEXT-INDEPENDENT EMBEDDING TO TRANSFORMER: EXPLORING SENTIMENT CLASSIFICATION IN ONLINE REVIEWS WITH DEEP LEARNING APPROACHES. *Journal of Theoretical and Applied Information Technology*, 15(19). www.jatit.org
- Yu, L., Zhou, R., Chen, R., & Lai, K. K. (2022). Missing Data Preprocessing in Credit Classification: One-Hot Encoding or Imputation? *Emerging Markets Finance and Trade*, 58(2), 472–482. <https://doi.org/10.1080/1540496X.2020.1825935>
- Zelaya, C. V. G. (2019). Towards Explaining the Effects of Data Preprocessing on Machine Learning. *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, 2086–2090. <https://doi.org/10.1109/ICDE.2019.00245>
- Zhang, W., Wei, W., Wang, W., Jin, L., & Cao, Z. (2021). Reducing BERT Computation by Padding Removal and Curriculum Learning. *Proceedings - 2021 IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2021*, 90–92. <https://doi.org/10.1109/ISPASS51385.2021.00025>

Zhuang, Y. ting, Wu, F., Chen, C., & Pan, Y. he. (2017). Challenges and opportunities: from big data to knowledge in AI 2.0. In *Frontiers of Information Technology and Electronic Engineering* (Vol. 18, Issue 1, pp. 3–14). Zhejiang University. <https://doi.org/10.1631/FITEE.1601883>